

Synapedia-Architekturspezifikation

Version 1.3 — Juli 2026

Ein vierschichtiges Lexikon für verankerte Konzeptdefinitionen

Substratschicht des Symbol Grounding Framework (SGF)

Status: Spezifikation

Datum: 2026-07-01

Autoren: James Lee Stäkelum

Lizenz: Creative Commons Attribution 4.0 International

Inhaltsverzeichnis

1. Präambel
2. Definitionen der Primitive
3. Postulate
4. Axiome
5. Theoreme
6. Metaphysische Festlegungen
7. Identitätskriterien
8. Deprecation- und Versionierungsprotokoll
9. Konfliktlösung für vererbte Ereignisskripte
10. Die vier Schichten
11. Die 15 semantischen Rollen
12. Kanonische IDs
13. Ereignisquellen und Autorität
14. Verankerung und Validierung
15. Wissenspakete

- 16. Umfang und Grenzen
 - 17. Prüftiefe als politische Entscheidung
 - 18. Verankerung nicht-physischer Entitäten
 - 19. Vergleich mit bestehenden Systemen
 - 20. Beispiele
 - 21. Proof-of-Concept-Durchlauf
 - 22. Anhänge
 - 23. Referenzen
-

Abschnitt 1: Präambel

1.1 Name

Dieses Dokument spezifiziert die **Synapedia-Architektur**, die Substratschicht des SGF.

1.2 Zweck

Synapedia **stellt** maschinenadressierbare Konzeptdefinitionen zum Zweck der Verankerung natürlichsprachlicher Aussagen in einem strukturierten, überprüfbaren Lexikon **bereit**. Es ersetzt die traditionelle Trennung von Lexikon und Ontologie durch eine einheitliche vierschichtige Struktur.

Einordnung: Synapedia ist ein **Lexikon** – eine neue Art maschinenlesbaren Wörterbuchs –, kein Wissensgraph. Es speichert keine Aussagen über die Welt. Es speichert Definitionen. Wahrheit, Falschheit und widersprüchliche Behauptungen gehören in die ABox (siehe Abschnitt 2.18).

Architektonische Innovation: Synapedia überwindet die traditionelle TBox/ABox-Trennung, indem es aktive, Hub-and-Spoke-Ereignisstrukturen direkt in Konzeptdefinitionen einbettet. In traditionellen Semantic-Web-Systemen war die TBox (Schema) eingefroren und konnte weder Zeit noch Veränderung abbilden, während die ABox (Instanzen) chaotisch und

unverankert war. Synapedia löst dies, indem es die Definitionsschicht selbst dynamisch macht – Ereignisse sind der TBox inhärent, nicht als Instanzdaten nachträglich hinzugefügt.

1.3 Umfang

Diese Spezifikation definiert:

- Die strukturellen Komponenten eines Synapedia-Eintrags (Knoten, Kanten, Schichten, Synapsen)
- Die Grammatik kanonischer IDs
- Die geschlossene Menge semantischer Rollen
- Die Postulate und Axiome, die den Graphen beschränken
- Die Validierungsregeln, denen Einträge genügen müssen
- Die Beziehung zwischen Synapedia und anderen SGF-Schichten (ABox, Wissenspakete, Frames)

Diese Spezifikation definiert **nicht**:

- Das Transportprotokoll für Synapedia-Einträge
- Die Abfragesprache für den Zugriff auf Synapedia-Daten
- Das Governance-Modell für die Autorenschaft von Einträgen
- Den Implementierungsfahrplan (siehe Anhang A)

1.4 Konformität

Eine Implementierung muss allen Axiomen (Abschnitt 4) und allen Postulaten (Abschnitt 3) entsprechen. Eine Implementierung sollte allen Validierungsregeln (Abschnitt 13) entsprechen. Eine Implementierung darf die Spezifikation nur durch Hinzufügen von Wissenspaketen erweitern; sie darf die Kernstruktur nicht verändern.

1.5 Dokumentkonventionen

- **Muss** kennzeichnet eine verbindliche Anforderung.

- **Sollte** kennzeichnet eine empfohlene Anforderung.
- **Darf** kennzeichnet eine optionale Funktion.
- **Darf nicht** kennzeichnet ein Verbot.

Abschnitt 2: Definitionen der Primitive

2.0 Was eine Definition ist (und was nicht)

Eine **Definition** in Synapedia ist die vollständige Menge struktureller Elemente – lexikalisch, ontologisch, mereologisch und perdurantistisch –, die ein Konzept innerhalb seiner Sprach- und Lemma-Geschwister-Menge eindeutig identifiziert. Eine Definition ist kein String. Sie ist ein Graph.

Lemma-Kollaps ist die systemische Unfähigkeit, verschiedene konzeptuelle Bedeutungen zuverlässig zu trennen, wenn sie denselben Oberflächentext teilen. Jedes Lexikon, dem ereignisverankerte Definitionen fehlen, leidet unter Lemma-Kollaps. Synapedia beseitigt ihn, indem es für jedes Konzept mit Lemma-Geschwistern eine perdurantistische Schicht vorschreibt.

Eine Synapedia-Definition tut Folgendes:

- Sie **verankert** das Konzept an einer Oberflächenform (Lemma, Wortart).
- Sie **positioniert** das Konzept innerhalb einer Typhierarchie (IS-A-Eltern).
- Sie **setzt** das Konzept zusammen, wenn die Zusammensetzung für die Identität tragend ist (Mereologie).
- Sie **unterscheidet** das Konzept von allen anderen, die sein Lemma teilen (perdurantistische Ereignisse).

Eine Synapedia-Definition tut Folgendes **nicht**:

- Sie liefert keinen natürlichsprachlichen Satz, der das Konzept „erklärt“ (eine Glosse ist optional, nicht definitorisch).
- Sie zählt nicht alle Eigenschaften des Konzepts auf (enzyklopädisches Wissen gehört in Wissenspakete).

- Sie liefert keine Wahrheitsbedingungen für Aussagen, die das Konzept verwenden (Wahrheit ist der Bereich der ABox).

Dies ist kein Glossar. In einem Glossar ist eine Definition ein Satz, der von einem Menschen gelesen werden kann. In Synapedia ist eine Definition eine maschinenadressierbare Graphstruktur mit vier Schichten, von denen jede unabhängig gegen die Axiome des Systems überprüfbar ist. Eine menschenlesbare Glosse darf der lexikalischen Schicht der Einfachheit halber beigefügt werden, aber sie ist nicht die Definition. Die Definition ist die Struktur.

2.1 Knoten

Ein **Knoten** ist die grundlegende Einheit des Synapedia-Graphen. Jeder Knoten hat genau eine kanonische ID (Abschnitt 2.3) und gehört zu genau einer Schicht (Abschnitt 2.6).

Einschränkung 2.1.1: Keine zwei Knoten dürfen dieselbe kanonische ID teilen.

Einschränkung 2.1.2: Jeder Knoten, der nicht im Prime Registry ist, muss mindestens eine ausgehende IS-A-Kante haben (Abschnitt 2.2.1).

Einschränkung 2.1.3: Jeder Knoten muss dem Axiom der Fundamentalen Verankerung genügen (Axiom II).

2.2 Kante

Eine **Kante** ist eine gerichtete, beschriftete Verbindung von einem Knoten (der Quelle) zu einem anderen Knoten (dem Ziel). Jede Kante hat genau einen Typ aus der in Tabelle 2.2-1 definierten geschlossenen Menge.

Tabelle 2.2-1: Zulässige Kantentypen

Kategorie	Typ	Richtung	Inverse
Ontologisch	IS-A	Kind → Elternteil	HAS-SUBCLASS (abgeleitet)

Mereologisch (Komponente)	HAS - COMPONENT	Ganzes → Teil	COMPONENT - OF
Mereologisch (Mitglied)	HAS - MEMBER	Sammlung → Mitglied	MEMBER - OF
Mereologisch (Portion)	HAS - PORTION	Masse → Portion	PORTION - OF
Ereignis (Synapsen-intern)	Die 15 Rollentypen	Siehe Abschnitt 11	Keine
Historisch/Spur	SUPERSEDE D - BY	veraltet → Ersatz	SUPERSEDES
Sprachübergreifen d	TRANSLATION - OF	Quelle → Ziel	TRANSLATION - OF (symmetrisch)

Entwurfshinweis 2.2.1 — IS-A-Subsumtionstypologie: Die IS-A-Beziehung umfasst mehrere Subsumtionsmodalitäten: taxonomisch (biologische Art), funktional (Zweck), beruflich (Tätigkeit), konstitutiv (Material) und mereologisch (Teil-von-Ganzem). Für alle wird ein einziger `IS-A` -Kantentyp verwendet; eine optionale `subsumption_type` -Annotation kann die Beziehung bei Bedarf verfeinern. Das Kernaxiom der Azyklizität gilt unabhängig vom Subtyp.

Einschränkung 2.2.1: Keine Kante darf einen Typ außerhalb dieser Menge haben. Dies ist die Invariante Kantenbeschränkung (Postulat III).

Einschränkung 2.2.2: `IS-A` -Kanten müssen einen gerichteten azyklischen Graphen bilden. Zyklen sind nicht erlaubt.

Einschränkung 2.2.3: `SUPERSEDED-BY` -Kanten dürfen keinen Zyklus bilden.

2.3 Kanonische ID

Eine **kanonische ID** ist ein global eindeutiger String-Identifikator, der der folgenden Grammatik entspricht:

```
canonical_id ::= "sgf:" language_tag "." lemma "." pos "." microgloss
language_tag ::= [a-z]{2} | [a-z]{3} | "xx"
lemma ::= [a-z][a-z0-9_]+
pos ::= "n" | "v" | "adj" | "adv" | "prep" | "conj" | "det" | "pron" |
microgloss ::= [a-z][a-z0-9_-]+
```

Beispiele: `sgf:en.wagon.n.horse_drawn_cargo` ,

`sgf:en.compose.v.create_music` , `sgf:en.beethoven.n.composer_1770` ,

`sgf:xx.prime.one.n.basic_primitive` .

Einschränkung 2.3.1: Eine kanonische ID ist nach ihrer Zuweisung unveränderlich. Sie darf nicht neu zugewiesen werden. Dies ist das Postulat der Referenzstabilität (Postulat IV).

Einschränkung 2.3.2: Die Mikroglosse-Komponente muss den Knoten innerhalb der Menge aller Knoten, die dieselbe language_tag, dasselbe lemma und dieselbe pos teilen, eindeutig identifizieren.

2.4 Lemma

Ein **Lemma** ist die kanonische Oberflächenform eines Wortes in einer gegebenen Sprache. Für Substantive ist das Lemma die nominativische Singularform. Für Verben ist das Lemma die Infinitivform. Für Adjektive ist das Lemma die Positivform. Ausnahmen sind für Sprachen zulässig, in denen diese Konventionen nicht gelten, dokumentiert in einem sprachspezifischen Anhang.

2.5 Mikroglosse

Eine **Mikroglosse** ist der kürzeste String, der diesen Knoten von allen anderen Knoten unterscheidet, die dasselbe Lemma und dieselbe Wortart innerhalb derselben Sprache teilen.

Regel 2.5.1: Eine Mikroglosse muss zwischen 1 und 4 Wörtern (einschließlich) lang sein, getrennt durch Unterstriche.

Regel 2.5.2: Eine Mikroglosse darf den Lemma-String nicht enthalten.

Regel 2.5.3: Eine Mikroglosse muss dem Axiom der Mikroglossenhinlänglichkeit genügen (Axiom IV).

2.6 Schicht

Eine **Schicht** ist eine von vier orthogonalen kategorialen Partitionen der Knotenmenge. Jeder Knoten gehört zu genau einer Schicht.

Tabelle 2.6-1: Die vier Schichten

Schicht	Primärer Inhalt	Kantentypen
LEXICAL	Lemma, POS, Glosse, Mikroglosse, Embedding	Keine (terminal)
ONTOLOGICAL	IS-A-Elternreferenzen	Nur IS-A
MEREOLOGICAL	Teil-Ganzes-Komposition	HAS-COMPONENT , HAS-MEMBER , HAS-PORION
PERDURANTIST	Minimale unterscheidende Synapsen	Nur Synapsen-interne Rollenkanten

Einschränkung 2.6.1: Kein Knoten darf zu mehr als einer Schicht gehören.

Einschränkung 2.6.2: Die Schicht eines Knotens wird bei der Erstellung bestimmt und ist unveränderlich.

2.7 Synapse

Eine **Synapse** ist ein strukturiertes Bündel von Kanten, das ein einzelnes Ereignis oder einen einzelnen Zustand repräsentiert. Sie besteht aus:

1. Einem VerbHub-Knoten (erforderlich, genau einer)
2. Einer Menge von Speichen (erforderlich, eine oder mehrere)
3. Optional einer Menge von Frame-Referenzen

Einschränkung 2.7.1: Eine Synapse muss mindestens eine Speiche und höchstens 15 Speichen enthalten (eine pro Rollentyp).

Einschränkung 2.7.2: Keine zwei Speichen in derselben Synapse dürfen denselben Rollentyp verwenden.

Einschränkung 2.7.3: Der VerbHub muss ein Knoten in der lexikalischen Schicht mit einer Verb-Wortart sein.

2.8 VerbHub

Ein **VerbHub** ist der zentrale Knoten einer Synapse. Er bestimmt den Ereignistyp und schränkt ein, welche Rollen verwendet werden dürfen. Der VerbHub muss ein Knoten mit der Wortart `v` sein und verankert sein (Axiom II).

2.9 Speiche

Eine **Speiche** ist eine einzelne Rollenkante innerhalb einer Synapse. Sie verbindet den VerbHub mit einem Teilnehmerknoten über genau einen Rollentyp. Der Teilnehmerknoten muss ein gültiger Synapedia-Knoten sein.

2.10 Frame

Ein **Frame** ist ein optionales Metadatenobjekt, das an eine Synapse angehängt wird, um ihre Interpretation zu verfeinern, ohne ihre Struktur zu verändern. Frames werden in Wissenspaketen definiert, nicht in Synapedia.

2.11 Verankerung

Verankerung ist die Eigenschaft eines Knotens, einen gültigen gerichteten Pfad über IS-A-Kanten entweder zu einem Knoten im Prime Registry oder zu einer festen Raumzeit-Koordinate zu haben.

Einschränkung 2.11.1: Jeder Knoten muss verankert sein (Axiom II).

2.12 Prime Registry

Das **Prime Registry** ist die Menge grundlegender Knoten, die gegeben, nicht definiert sind. Die Menge ist nicht leer (Postulat I). Prime-Registry-Knoten haben das Sprach-Tag `xx` und haben keine IS-A-Eltern.

2.13 Wissenspaket

Ein **Wissenspaket** ist ein signiertes, versioniertes Bündel von SGF-Objekten – Synapsen, Gruppen, Frames –, die über ihre kanonischen IDs an Synapedia-Knoten angehängt werden. Wissenspakete verändern keine Synapedia-Einträge.

Einschränkung 2.13.1: Ein Wissenspaket darf keine Knoten enthalten, die behaupten, in einer Synapedia-Schicht zu sein.

Einschränkung 2.13.2: Ein Wissenspaket darf keine `IS-A` -Kanten zum Synapedia-Graphen hinzufügen.

2.14 Lemma-Geschwister

Zwei Knoten sind **Lemma-Geschwister**, wenn sie dasselbe Lemma und dieselbe Wortart innerhalb desselben Sprach-Tags teilen.

Einschränkung 2.14.1: Keine zwei Lemma-Geschwister dürfen dieselbe Mikroglosse teilen.

Einschränkung 2.14.2: Keine zwei Lemma-Geschwister dürfen identische perdurantistische Schichten haben. Wenn sie dies tun, sind sie dasselbe Konzept und müssen zusammengeführt werden.

2.15 SELF-Referenz

`SELF` ist ein reservierter Referenzoperator, der innerhalb der perdurantistischen Schicht eines Knotens verwendet wird. Er löst sich zur kanonischen ID des enthaltenden Knotens auf.

Einschränkung 2.15.1: `SELF` darf nur in der perdurantistischen Schicht eines Knotens erscheinen. Es darf in keinem anderen Kontext erscheinen.

Einschränkung 2.15.2: `SELF` ist kein Knoten. Es hat keine kanonische ID, keine Schicht und keine Eigenschaften.

Einschränkung 2.15.3: Wenn eine Synapse, die `SELF` enthält, von einem Kindknoten (über IS-A) geerbt wird, löst sich `SELF` zum Kindknoten auf, nicht zum Elternteil.

2.16 Zeitkoordinate

Eine **Zeitkoordinate** ist ein Wert, der in der `HAS_TIME`-Rolle verwendet wird. Er muss einer der folgenden Typen sein:

- **Punkt:** Ein ISO-8601-Datetime-String. Die Genauigkeit kann variieren.
- **Intervall:** Ein Tupel `(start, end)` im ISO-8601-Format.
- **Wiederholungsmuster:** Ein String in einem definierten Wiederholungsformat (z.B. `"every_spring"`).
- **Kalendarische Referenz:** Eine Knotenreferenz auf ein zeitbezogenes Konzept.

Einschränkung 2.16.1: Eine Zeitkoordinate muss sich zum Abfragezeitpunkt in eine bestimmte Zeit oder einen bestimmten Bereich auflösen.

Einschränkung 2.16.2: Zeitkoordinaten werden durch ihre Beziehung zu standardisierten Zeitskalen verankert, die im Prime Registry über `time.n.physical_dimension` verankert sind.

2.17 Ereignisidentität

Zwei Synapsen sind dasselbe **Ereignis** genau dann, wenn alle folgenden Bedingungen erfüllt sind:

1. **Gleicher VerbHub:** Beide Synapsen verwenden denselben VerbHub-Knoten (dieselbe kanonische ID).

2. **Gleiche Rollen-Teilnehmer-Zuordnung:** Für jede Rolle, die in beiden Synapsen vorkommt, sind die Teilnehmerknoten identisch.
3. **Gleiche Zeit:** Beide Synapsen haben denselben `HAS_TIME` -Wert, oder keine gibt einen an.
4. **Gleicher Ort:** Wenn beide `HAS_LOCATION` angeben, sind die Orte derselbe Knoten.

Einschränkung 2.17.1: Ereignisidentität erfordert keine identischen Frames. Zwei Synapsen mit unterschiedlichen Frame-Interpretationen sind dasselbe Ereignis, wenn der strukturelle Kern übereinstimmt.

Einschränkung 2.17.2: Zwei Synapsen, die in der Struktur identisch sind, aber in verschiedenen Wissenspaketen erscheinen, beziehen sich auf dasselbe Ereignis, sofern nicht explizit als unterschiedlich gekennzeichnet.

2.18 ABox (Assertional Box)

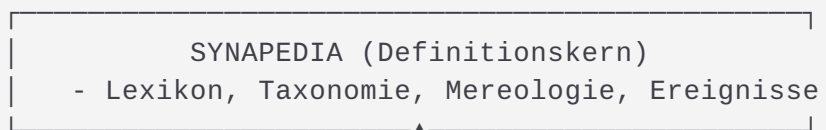
Die **ABox** ist die Schicht des SGF, die Behauptungen speichert – Aussagen über die Welt. Behauptungen können wahr, falsch, irrtümlich, bewusst täuschend, widersprüchlich oder hypothetisch sein. Die ABox entscheidet nicht über Wahrheit. Sie speichert, was gesagt wurde, von wem und unter welchen Bedingungen.

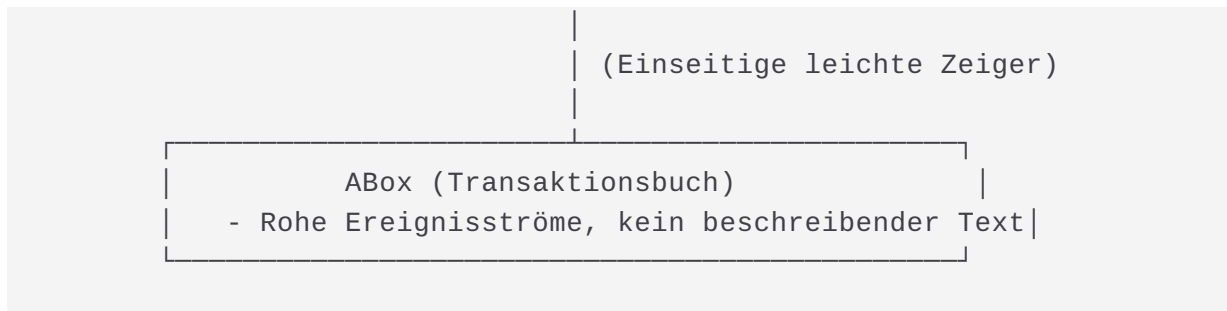
Einschränkung 2.18.1: Die ABox referenziert Synapedia-kanonische IDs für alle Konzeptreferenzen. Sie definiert keine Konzepte.

Einschränkung 2.18.2: Synapedia referenziert die ABox nicht. Die Beziehung ist einseitig.

Entwurfshinweis 2.18.1 — Asymmetrischer Abhängigkeitsvektor:

Die Beziehung zwischen Synapedia und der ABox ist strikt einseitig:





Die ABox enthält keine beschreibenden Metadaten. Sie speichert nur skeletthafte Drahtrahmen – kanonische IDs und Rollenzuweisungen – und hydriert bei Bedarf Bedeutung durch Referenzierung von Synapedia.

Einschränkung 2.18.3: Eine Behauptung in der ABox kann durch eine andere Behauptung widersprochen werden. Dies ist kein Fehler. Wahrheit ist eine Verhandlung, die oberhalb beider Schichten stattfindet.

Beispiel einer ABox-Synapse:

```
{
  "id": "abox:claim.20260628.001",
  "assertion": {
    "subject": "sgf:en.defendant.n.person_accused",
    "predicate": "sgf:en.drive.v.operate_vehicle",
    "object": "sgf:en.car.n.motor_vehicle",
    "manner": "sgf:en.speed_limit.n.legal_maximum"
  },
  "attribution": {
    "source": "testimony_001",
    "timestamp": "2026-06-28T14:30:00Z",
    "confidence": "claimed"
  }
}
```

2.19 Die orthogonale Dualachsen-Aufteilung

Synapedia organisiert Bedeutung über zwei unabhängige geometrische Ebenen:

- **Y-Achse (Objekt-/Materiallogik):** Steuert die strukturelle Abstammung (IS-A) und die physische oder konzeptuelle Zusammensetzung (HAS-

COMPONENT, HAS-MEMBER, HAS-PORION). Diese Achse verfolgt, was Dinge *sind* und wie sie *gebaut* sind, unabhängig von Handlung oder Zeit. Sie umfasst die lexikalische, ontologische und mereologische Schicht.

- **X-Achse (Ereignis-/Handlungslogik):** Steuert dynamische Veränderungen, Verhaltensweisen und situative Vorkommnisse unter Verwendung eines zentralisierten VerbHubs, der mit strahlenden Speichen verbunden ist. Diese Achse verfolgt, was Dinge *tun* und was mit ihnen geschieht. Sie umfasst die perdurantistische Schicht.

Die beiden Achsen sind orthogonal. Die Identität eines Konzepts ist nur dann vollständig spezifiziert, wenn beide Achsen aufgelöst sind. Ein Wagen ist nicht vollständig durch seine Y-Achsen-Position definiert (Fahrzeug, Behälter, Räder, Achse) – er benötigt seine X-Achsen-Position (transportiert, zieht, rollt), um von einem Karren oder einem Schlitten unterschieden zu werden.

Einschränkung 2.19.1: Jeder Knoten muss mindestens eine besetzte Achse haben. Ein Knoten mit nur einer lexikalischen Schicht (keine IS-A-Eltern, keine Mereologie, keine Ereignisse) ist nur erlaubt, wenn er sich im Prime Registry befindet.

Abschnitt 3: Postulate

Postulate sind konstruktionale Annahmen. Sie werden innerhalb des Systems nicht bewiesen.

Postulat I — Existenz von Primitiven

Es existiert eine nicht-leere Menge von Knoten, genannt das Prime Registry. Alle anderen Knoten leiten ihre Verankerung von Pfaden zu dieser Menge ab.

Postulat II — Kategorizität

Jeder Knoten gehört zu genau einer der vier Schichten. Kein Knoten darf zu mehr als einer Schicht gehören.

Postulat III — Die Invariante Kantenbeschränkung (Kantenexklusivität)

Die zulässigen Kantentypen sind genau die in Tabelle 2.2-1 aufgeführten. Keine Kante außerhalb dieser Menge darf existieren. Keine Kante eines Typs darf als eine andere interpretiert werden.

Während der Wortschatz von Nomen und Verben unendlich offen ist, sind die relationalen Verbindungen, die einen Ereignis-Hub mit seinen Teilnehmern verbinden, dauerhaft auf genau 15 orthogonale semantische Rollen beschränkt. Diese Einschränkung ist invariant. Sie kann durch kein Wissenspaket oder nachgelagertes System erweitert, modifiziert oder außer Kraft gesetzt werden.

Postulat IV — Referenzstabilität

Eine kanonische ID ist nach der Zuweisung unveränderlich. Sie darf nicht einem anderen Konzept neu zugewiesen werden.

Postulat V — Polyhierarchie-Erlaubnis

Ein Knoten darf null oder mehr IS-A-Eltern haben. Es wird keine Obergrenze auferlegt. Kein Elternteil ist primär.

Postulat VI — Ereignisabgeschlossenheit

Alle Ereignisinformationen in Synapedia werden unter Verwendung genau der 15 in Abschnitt 11 definierten Rollentypen ausgedrückt. Die Menge ist abgeschlossen.

Postulat VII — Graph-Endlichkeit

Der Synapedia-Graph enthält eine endliche Anzahl von Knoten und Kanten. Alle Graphalgorithmen terminieren in endlicher Zeit.

Postulat VIII — Sprachunabhängigkeit

Der strukturelle Inhalt eines Knotens (IS-A-Eltern, mereologische Kanten, Synapsen) ist sprachunabhängig. Zwei Knoten in verschiedenen Sprachen, die sich auf dasselbe Konzept beziehen, sind durch eine `TRANSLATION-OF` -Kante verbunden.

Abschnitt 4: Axiome

Jedes Axiom ist eine logische Einschränkung, der alle Einträge genügen müssen.

Axiom I — Ontologische Azyklizität

Der gerichtete Graph, der von allen `IS-A` -Kanten gebildet wird, ist strikt azyklisch.

Axiom II — Fundamentale Verankerung

Jeder Knoten muss einen Pfad über `IS-A` -Kanten entweder zu einem Prime-Registry-Knoten oder einer festen Raumzeit-Koordinate haben.

Axiom III — Komponententransitivität

Die `HAS-COMPONENT` -Beziehung über die Komponente-Integrale-Ganzes-Achse ist transitiv. Diese Transitivität erstreckt sich nicht auf `HAS-MEMBER` oder `HAS-PORION` .

Axiom IV — Mikroglossenhinlänglichkeit

Innerhalb der Menge aller Knoten, die dasselbe Sprach-Tag, dasselbe Lemma und dieselbe Wortart teilen, dürfen keine zwei Knoten dieselbe Mikroglosse teilen. Die Funktion `microgloss: L → String` ist injektiv.

Axiom V — Ereignisminimalität

Keine Synapse in der perdurantistischen Schicht eines Knotens darf entfernt werden, während die Unterscheidung von allen anderen Knoten, die dasselbe Lemma und dieselbe Wortart teilen, erhalten bleibt.

Axiom VI — Identitätseindeutigkeit

Keine zwei verschiedenen Knoten dürfen dieselbe kanonische ID teilen.

Abschnitt 5: Theoreme

Theorem I — Verankerungsketten sind endlich

Jede Verankerungskette im Synapedia-Graphen hat endliche Länge.

Beweis: Nach Axiom II hat jeder Knoten einen Pfad zu einem Prime-Registry-Knoten oder einer Raumzeit-Koordinate. Nach Axiom I ist der Graph azyklisch. Nach Postulat VII ist der Graph endlich. Daher ist jeder Pfad endlich.

Theorem II — Lemma-Geschwister-Trennung

Für zwei beliebige verschiedene Lemma-Geschwister-Knoten unterscheiden sich ihre perdurantistischen Schichten in mindestens einer Synapse.

Beweis: Nach Axiom IV unterscheiden sich ihre Mikroglossen. Nach Axiom V ist jede perdurantistische Schicht minimal. Wären die Schichten identisch, müssten sich die Mikroglossen allein aufgrund lexikalischer Kriterien unterscheiden, was der Definition der Mikroglosse als Disambiguierungsschlüssel widerspricht. Daher müssen sich die Schichten unterscheiden.

Theorem III — Keine verwaisten Definitionen

Jeder Knoten, der nicht im Prime Registry ist, hat mindestens eine ausgehende `IS-A` -Kante.

Beweis: Nach Axiom II muss jeder Knoten einen Pfad zum Prime Registry über `IS-A` -Kanten haben. Ein Knoten ohne ausgehende `IS-A` -Kanten hätte keinen solchen Pfad, es sei denn, er wäre selbst ein Prime-Registry-Knoten.

Theorem IV — Keine ID-Kollisionen

Keine zwei Knoten dürfen dieselbe kanonische ID teilen. Direkt aus Axiom VI.

Theorem V — Deprecation bricht Verankerung nicht

Wenn ein Knoten als veraltet markiert und über `SUPERSEDED-BY` mit einem Ersatz verknüpft wird, bleibt der veraltete Knoten verankert. Die `SUPERSEDED-BY` -Kante ist keine `IS-A` -Kante; bestehende `IS-A` -Pfade bleiben unverändert.

Abschnitt 6: Metaphysische Festlegungen

Diese Festlegungen beschreiben die philosophische Haltung der Architektur. Sie werden nicht vom System erzwungen, sondern motivieren sein Design.

6.1 Perdurantismus über Endurantismus

Ereignisse sind Bürger erster Klasse. Ein Konzept wird nicht nur dadurch definiert, was es ist, sondern auch dadurch, was es tut und was ihm widerfährt. Die perdurantistische Schicht ist tragend, nicht dekorativ.

6.1.1 Warum „Perdurantistisch“?

Der Begriff ist der philosophischen Debatte über Persistenz durch die Zeit entlehnt.

- **Endurantismus** besagt, dass ein Objekt in jedem Moment seiner Existenz vollständig vorhanden ist. Ein Wagen *ist* in jedem Augenblick ein Wagen – dasselbe vierrädrige Ding. Veränderung geschieht mit ihm, aber das Objekt selbst hat keine zeitlichen Teile.

- **Perdurantismus** besagt, dass ein Objekt über die Zeit verteilt ist und zeitliche Teile (oder „Stadien“) hat. Ein Wagen ist nicht nur das statische Ding in einem Moment – er ist die Summe all seiner Stadien, einschließlich der Ereignisse, an denen er teilnimmt: gezogen werden, rollen, Fracht transportieren. Der Wagen *perduziert* durch diese Ereignisse.

Die perdurantistische Schicht von Synapedia erfasst die Ereignisse, Prozesse und Verhaltensweisen, die ein Konzept definieren – die zeitlichen Teile, ohne die das Konzept nicht von seinen Lemma-Geschwistern unterschieden werden kann. Ein Wagen ist nicht nur ein statisches Fahrzeug mit Rädern; er *perduziert* durch das Gezogenwerden, Rollen und Transportieren von Fracht. Der Name signalisiert, dass unsere Definitionen dynamisch, nicht statisch sind.

Die anderen drei Schichten (lexikalisch, ontologisch, mereologisch) beschreiben, was das Konzept *in einer einzelnen Momentaufnahme* ist – sein Name, seine Kategorien, seine Teile. Die perdurantistische Schicht beschreibt, was das Konzept *tut* – seine wesentlichen Ereignisse, die Rollen, die es spielt, die Handlungen, die es zu *der* Art von Ding machen und nicht zu einer anderen.

6.2 Konzeptueller Realismus

Konzepte existieren unabhängig von ihren sprachlichen Etiketten. Zwei Knoten in verschiedenen Sprachen, die sich auf dasselbe Konzept beziehen, teilen eine TRANSLATION-OF -Kante. Das Konzept selbst ist an keine Sprache gebunden.

6.3 Minimalismus

Eine Definition ist die kleinste Menge von Tatsachen, die ein Konzept innerhalb seiner Lemma-Geschwister-Menge eindeutig identifiziert. Enzyklopädisches Wissen ist keine Definition.

6.4 Fallibilismus

Definitionen können falsch sein. Sie können durch Deprecation und Ersetzung korrigiert werden. Es werden keine Informationen vernichtet; die historische Aufzeichnung bleibt erhalten.

6.5 Verankerung

Jedes Konzept muss letztlich entweder auf ein Primitiv (Prime Registry) oder eine raumzeitliche Lokation zurückführbar sein. Es gibt keine schwebenden Definitionen.

6.6 Pluralismus

Ein Konzept kann mehrere gültige IS-A-Eltern haben. Das System erzwingt keinen einzigen taxonomischen Baum.

6.7 Behauptungsagnostizismus

Das System entscheidet nicht über Wahrheit. Synapedia definiert Konzepte; die ABox speichert Behauptungen. Wahrheit, Falschheit, Widerspruch und Täuschung sind alle in der ABox gültig. Das System ist dem Wahrheitswert von Aussagen gegenüber strukturell neutral.

Abschnitt 7: Identitätskriterien

7.1 Die Identitätsregel

Zwei Kandidateneinträge beziehen sich genau dann auf dasselbe Konzept, wenn alle folgenden Bedingungen erfüllt sind:

1. Gleiches Lemma
2. Gleiche Wortart
3. Gleiches Sprach-Tag
4. Gleiche Menge minimal unterscheidender perdurantistischer Synapsen

7.2 Die Ein-Eintrag-Regel

Es muss genau einen Eintrag in Synapedia für jedes distincte Konzept innerhalb einer gegebenen Sprache geben.

7.3 Synonymie und sprachübergreifende Identität

Zwei Einträge in verschiedenen Sprachen, die sich auf dasselbe Konzept beziehen, sind nicht derselbe Eintrag. Sie haben unterschiedliche kanonische IDs und sind durch eine `TRANSLATION-OF`-Kante verbunden.

7.4 Lemma-Geschwister-Distinktheit

Zwei Lemma-Geschwister müssen sich in mindestens einem der folgenden Punkte unterscheiden: Mikroglosse oder einer oder mehreren Synapsen in der perdurantistischen Schicht.

Abschnitt 8: Deprecation- und Versionierungsprotokoll

8.1 Unveränderlichkeit kanonischer IDs

Einmal zugewiesen, ist eine kanonische ID unveränderlich. Sie ändert nie ihre Bedeutung.

8.2 Korrekturprotokoll

Schritt 1: Markiere den vorhandenen Knoten mit Status `DEPRECATED`. Erstelle einen Deprecation-Datensatz mit Grund, Zeitstempel und Autorität.

Schritt 2: Erstelle einen neuen Knoten mit einer neuen kanonischen ID, die die korrigierte Definition enthält.

Schritt 3: Füge eine `SUPERSEDED-BY`-Kante vom veralteten Knoten zum Ersatz hinzu.

Schritt 4: Benachrichtige alle registrierten Wissenspakete, die die veraltete ID referenzieren.

8.3 Versionierung

Synapedia als Ganzes wird auf Graphenebene versioniert. Jede Version wird durch einen Hash der gesamten Knoten- und Kantenmenge identifiziert. Kleinere Korrekturen an Glossen oder Embeddings, die die Graphstruktur nicht beeinflussen, können ohne Versionsinkrement vorgenommen werden.

8.4 Rückwärtskompatibilität

Veraltete IDs werden nie entfernt. Jedes System, das auf eine veraltete ID angewiesen war, wird weiterhin funktionieren. Der Graph ist auf Knotenebene append-only.

Abschnitt 9: Konfliktlösung für vererbte Ereignisskripte

9.1 Das Problem

Wenn ein Knoten von mehreren IS-A-Eltern erbt, können diese Eltern widersprüchliche Synapsen in ihren perdurantistischen Schichten definieren.

9.2 Lösungsregeln

Wende die folgenden Regeln in der angegebenen Reihenfolge an. Die erste Regel, die den Konflikt löst, wird verwendet.

Regel 1 — Explizite Überschreibung: Wenn der Knoten selbst eine Synapse definiert, die einer geerbten Synapse direkt widerspricht, gewinnt die eigene Definition des Knotens.

Regel 2 — Spezifität: Wenn zwei Eltern widersprüchliche Synapsen definieren, hat der Elternteil, der tiefer in der IS-A-Hierarchie steht, Priorität. Die Tiefe wird als Länge des längsten Pfades vom Elternteil zum Prime Registry gemessen.

Regel 3 — Zeitliche Priorität: Wenn zwei Eltern auf derselben Tiefe widersprüchliche Synapsen definieren, hat der Elternteil Priorität, dessen Knoten kürzlicher erstellt wurde.

Regel 4 — Manuelle Annotation: Wenn keine der obigen Regeln ein zufriedenstellendes Ergebnis liefert, darf ein menschlicher Annotator eine `CONFLICT_RESOLUTION` -Annotation hinzufügen, die alle automatischen Lösungen außer Kraft setzt.

9.3 Nicht-konfliktäre Vererbung

Wenn zwei Eltern unterschiedliche Synapsen definieren, die nicht denselben VerbHub teilen, liegt kein Konflikt vor. Beide werden vererbt.

Abschnitt 10: Die vier Schichten

10.1 Lexikalische Schicht

10.1.1 Felder

Feld	Typ	Erforderlich	Beschreibung
<code>canonical_id</code>	String	Erforderlich	Muss Abschnitt 2.3 entsprechen
<code>layer</code>	String	Erforderlich	Muss "LEXICAL" sein
<code>lemma</code>	String	Erforderlich	Kanonische Oberflächenform
<code>pos</code>	String	Erforderlich	Einer der definierten POS-Tags
<code>microgloss</code>	String	Erforderlich	Kurzer Disambiguierungsstring
<code>gloss</code>	String	Optional	Natürlichsprachliche Definition (ein Satz)

<code>embedding</code>	Float[]	Optional	Vektoreinbettung
<code>examples</code>	String []	Optional	Beispielsätze
<code>sourcing</code>	Objekt	Optional	Quellmetadaten (siehe Abschnitt 13)

10.1.2 Einschränkungen

Einschränkung 10.1.1: Ein lexikalischer Knoten darf keine ausgehenden Kanten haben.

Einschränkung 10.1.2: Ein lexikalischer Knoten muss eine nicht-leere Mikroglosse haben.

10.2 Ontologische Schicht

10.2.1 Felder

Feld	Typ	Erforderlich	Beschreibung
<code>canonical_id</code>	String	Erforderlich	Muss Abschnitt 2.3 entsprechen
<code>layer</code>	String	Erforderlich	Muss "ONTOLOGICAL" sein
<code>is_a</code>	String []	Erforderlich	Array von Eltern-Canonical-IDs
<code>grounding_status</code>	Enum	Erforderlich	"GROUNDED" oder "UNRESOLVED"

10.2.2 Einschränkungen

Einschränkung 10.2.1: Jede ID im `is_a`-Array muss einem vorhandenen Knoten entsprechen.

Einschränkung 10.2.2: Die `is_a` -Kanten dürfen keinen Zyklus erzeugen (Axiom I).

Einschränkung 10.2.3: Wenn `grounding_status` "UNRESOLVED" ist, muss der Knoten als provisorisch markiert werden.

10.3 Mereologische Schicht

10.3.1 Felder

Feld	Typ	Erforderlich	Beschreibung
<code>canonical_id</code>	String	Erforderlich	Muss Abschnitt 2.3 entsprechen
<code>layer</code>	String	Erforderlich	Muss "MEREOLOGICAL" sein
<code>has_component</code>	String[]	Optional	Komponententeil-IDs
<code>has_member</code>	String[]	Optional	Mitglieds-IDs
<code>has_portion</code>	String[]	Optional	Portions-IDs

10.3.2 Einschränkungen

Einschränkung 10.3.1: Ein Knoten muss mindestens ein mereologisches Feld besetzt haben, um zu dieser Schicht zu gehören.

Einschränkung 10.3.2: Die `has_component` -Beziehung ist transitiv (Axiom III). Die `has_member` - und `has_portion` -Beziehungen sind nicht transitiv.

Entwurfshinweis 10.3.1 — Betriebliche Konsequenzen der mereologischen Transitivität:

Da HAS-COMPONENT transitiv ist, pflanzt sich eine Änderung an einer Komponente nach oben fort. Wenn ein Wagen seine Räder verliert, berechnet

die mereologische Regel automatisch, dass der Funktionsstatus des Ganzen beeinträchtigt ist – ohne dass ein externes Sensorereignis den Ausfall melden muss. Dies ermöglicht deterministisches Schließen über den Systemzustand allein aus strukturellen Informationen.

Einschränkung 10.3.3: Mereologie ist optional. Die meisten abstrakten Konzepte und Personen benötigen sie nicht.

10.4 Perdurantistische Schicht

10.4.1 Felder

Feld	Typ	Erforderlich	Beschreibung
<code>canonical_id</code>	String	Erforderlich	Muss Abschnitt 2.3 entsprechen
<code>layer</code>	String	Erforderlich	Muss "PERDURANTIST" sein
<code>essential_events</code>	Synapse[]	Erforderlich	Array von Synapsen (mindestens 1, außer wenn keine Lemma-Geschwister existieren)
<code>contingent_events</code>	Synapse[]	Optional	Historisch bedeutsam, aber nicht definitorisch
<code>sourcing</code>	Objekt	Optional	Quellmetadaten (siehe Abschnitt 13)

10.4.2 Einschränkungen

Einschränkung 10.4.1: Das `essential_events` -Array muss mindestens eine Synapse enthalten, es sei denn, der Knoten hat keine Lemma-Geschwister.

Einschränkung 10.4.2: Jede Synapse in `essential_events` muss dem Axiom der Ereignisminimalität genügen (Axiom V).

Einschränkung 10.4.3: Das `contingent_events` -Array unterliegt nicht dem Minimalitätsaxiom.

10.5 Verben und die vier Schichten

Verben sind Knoten erster Klasse, die derselben vierschichtigen Architektur mit spezifischen Konventionen folgen.

10.5.1 Lexikalische Schicht (Verb)

Standardfelder. Die Mikroglosse für Verben sollte den semantischen Kernrahmen kodieren: `compose.v.create_music` , `pull.v.exert_force` .

10.5.2 Ontologische Schicht (Verb)

Verben haben IS-A-Eltern in einer Verbhierarchie, die nach semantischen Domänen organisiert ist:

- `compose.create_music` IS-A `create.v.bring_into_existence`
- `create.v.bring_into_existence` IS-A `act.v.do_something`
- `act.v.do_something` IS-A `event.v.happen`

Einschränkung 10.5.1: Verb-IS-A-Kanten implizieren Ereignissubsumtion: Wenn X IS-A Y, dann ist jedes Ereignis vom Typ X auch ein Ereignis vom Typ Y.

10.5.3 Mereologische Schicht (Verb)

Verben haben typischerweise keine Mereologie.

10.5.4 Perdurantistische Schicht (Verb)

Die perdurantistische Schicht eines Verbs definiert seine **Rolleneinschränkungen** – die minimale Menge von Rollen, die für eine wohlgeformte Synapse erforderlich sind.

Beispiel — `compose.v.create_music` :

```
{
  "canonical_id": "sgf:en.compose.v.create_music",
  "layer": "perdurantist",
  "lexical": { "lemma": "compose", "pos": "v", "microgloss": "create_m",
  "ontological": { "is_a": ["sgf:en.create.v.bring_into_existence"] },
  "perdurantist": {
    "core_roles": ["HAS_AGENT", "HAS_THEME"],
    "permitted_roles": ["HAS_TIME", "HAS_LOCATION", "HAS_INSTRUMENT",
    "essential_events": []
  }
}
```

Einschränkung 10.5.2: Eine Synapse, die einen bestimmten VerbHub verwendet, muss alle in `core_roles` aufgeführten Rollen enthalten.

Einschränkung 10.5.3: Eine Synapse, die einen bestimmten VerbHub verwendet, darf keine Rolle enthalten, die nicht in `core_roles` oder `permitted_roles` aufgeführt ist.

Einschränkung 10.5.4: Verbknoten dürfen ein leeres `essential_events` - Array haben. Die Definition des Verbs erfolgt durch seine Rolleneinschränkungen, nicht durch Ereignisse über das Verb selbst.

10.6 Zustandskonzepte — Adjektive und Eigenschaften

Zustandskonzepte – Adjektive, Adverbien und andere Eigenschaftsbegriffe – erfordern eine andere Behandlung in der perdurantistischen Schicht.

10.6.1 Lexikalische Schicht (Adjektiv/Adverb)

Standard lexikalische Schicht. Die Wortart ist `adj` oder `adv`.

10.6.2 Ontologische Schicht (Adjektiv)

Adjektive haben IS-A-Eltern in einer Eigenschaftshierarchie:

- `red.adj.color_red` IS-A `color.adj.chromatic_property` IS-A `property.adj.perceptual_attribute` IS-A `quality.n.abstract_attribute`

10.6.3 Perdurantistische Schicht (Adjektiv)

Adjektive definieren **charakteristische Situationen**, in denen die Eigenschaft manifestiert wird, anstatt Ereignisse im üblichen Sinne.

Beispiel — `red.adj.color_red`:

```
{
  "canonical_id": "sgf:en.red.adj.color_red",
  "layer": "perdurantist",
  "lexical": { "lemma": "red", "pos": "adj", "microgloss": "color_red" },
  "ontological": { "is_a": ["sgf:en.color.adj.chromatic_property"] },
  "perdurantist": {
    "characteristic_situations": [
      {
        "hub": "sgf:en.appear.v.present_visually",
        "spokes": {
          "HAS_THEME": ["sgf:en.ripe_tomato.n.object_with_property"],
          "HAS_ATTRIBUTE": ["SELF"]
        }
      }
    ]
  }
}
```

Einschränkung 10.6.1: Ein Knoten mit POS `adj` oder `adv` muss `characteristic_situations` anstelle von `essential_events` verwenden.

Einschränkung 10.6.2: Ein Knoten mit POS `adj` oder `adv` darf `essential_events` nicht verwenden.

Einschränkung 10.6.3: Dasselbe Axiom der Ereignisminimalität (Axiom V) gilt: Keine charakteristische Situation darf entfernt werden, während die Unterscheidung von Lemma-Geschwistern erhalten bleibt.

Abschnitt 11: Die 15 semantischen Rollen

11.1 Zweck

Die 15 semantischen Rollen bilden die geschlossene Grammatik für alle Ereignisrepräsentationen. Sie sind die einzig zulässigen Rollentypen für Synapsenspeichen.

11.2 Rollendefinitionen

#	Rollenname	Kategorie	Definition	Beispiel
1	HAS_AGENT	Kern	Bewusster Initiator der Handlung. Muss empfindsam sein.	„Er fuhr den Wagen.“ → AGENT: er
2	HAS_PATIENT	Kern	Entität, die eine Zustandsänderung durchläuft.	„Er brach die Achse.“ → PATIENT: Achse
3	HAS_THEME	Kern	Entität, die bewegt, lokalisiert oder gehalten wird. Keine Zustandsänderung.	„Er lud Getreide auf den Wagen.“ → THEME: Getreide
4	HAS_EXPERIENCER	Kern	Entität, die nicht-bewusst erfährt.	„Er fühlte den Wagen ruckeln.“ → EXPERIENCER: er
5	HAS_RECIPIENT	Kern	Entität, die das Thema erhält.	„Er gab die Zügel dem Fuhrmann.“ → RECIPIENT: Fuhrmann
6	HAS_BENEFICIARY	Kern	Entität, zu deren Nutzen die	„Er baute einen Wagen für den Bauern.“ →

			Handlung ausgeführt wird.	BENEFICIARY: Bauer
7	HAS_TIME	Umstand	Zeitkoordinate des Ereignisses.	„Er fuhr im Morgengrauen ab.“ → TIME: Morgengrauen
8	HAS_LOCATION	Umstand	Räumliche Region, in der das Ereignis stattfindet.	„Er parkte den Wagen nahe der Scheune.“ → LOCATION: Scheune
9	HAS_SOURCE	Umstand	Ausgangszustand oder -ort des Themas.	„Er fuhr vom Bauernhof.“ → SOURCE: Bauernhof
10	HAS_DESTINATION	Umstand	Endzustand oder -ort des Themas.	„Er fuhr zum Markt.“ → DESTINATION: Markt
11	HAS_MANNER	Umstand	Art und Weise, wie das Ereignis ausgeführt wird.	„Er ritt den Wagen langsam.“ → MANNER: langsam
12	HAS_INSTRUMENT	Umstand	Nicht- empfindsames Werkzeug zur Ausführung der Handlung.	„Er spannte die Pferde mit einem Geschirr an.“ → INSTRUMENT: Geschirr
13	HAS_CAUSE	Umstand	Unbelebter Auslöser einer	„Die Achse brach durch das

			Zustandsänderung.	Gewicht." → CAUSE: Gewicht
1 4	HAS_REASON	Umstand	Einheitlicher motivationaler Grund (Grund + Zweck).	„Er ging zum Markt, um Getreide zu verkaufen." → REASON: Getreide_verkaufen
1 5	HAS_ATTRIBUTE	Umstand	Ereignisresultierende Eigenschaft, die einem Teilnehmer zugewiesen wird.	„Er strich den Wagen rot." → ATTRIBUTE: rot

11.3 Einschränkungen

Einschränkung 11.3.1: Keine Synapse darf einen Rollentyp außerhalb dieser Menge verwenden.

Einschränkung 11.3.2: Keine zwei Speichen in derselben Synapse dürfen denselben Rollentyp verwenden.

Einschränkung 11.3.3: Eine Umstandsrolle darf weggelassen werden. Eine Kernrolle darf nur weggelassen werden, wenn der Ereignistyp sie logisch nicht erfordert.

Einschränkung 11.3.4: HAS_AGENT darf nur mit einem empfindsamen Knoten verbunden werden.

Einschränkung 11.3.5: HAS_INSTRUMENT darf nur mit einem nicht-empfindsamen Knoten verbunden werden.

Einschränkung 11.3.6: HAS_CAUSE muss mit einem nicht-empfindsamen Knoten oder einer Naturkraft verbunden werden. Für bewusste Handlungen

verwende `HAS_REASON` .

11.4 Grund-Zweck-Vereinheitlichung

`HAS_REASON` ist eine vereinheitlichte Rolle, die sowohl rückwärtsgerichtetes Motiv als auch vorwärtsgerichteten Zweck abdeckt. Wenn die Unterscheidung notwendig ist, verfeinern Frames die Interpretation. Die Rollengrammatik bleibt stabil; die Nuance lebt im Frame.

Abschnitt 12: Kanonische IDs

12.1 Grammatik

Bereits in Abschnitt 2.3 spezifiziert. Dieser Abschnitt enthält zusätzliche Regeln.

12.2 Zuweisungsregeln

Regel 12.2.1: Eine kanonische ID wird bei der Knotenerstellung zugewiesen. Sie wird nie geändert.

Regel 12.2.2: Die Mikroglosse muss so gewählt werden, dass sie unter Lemma-Geschwistern eindeutig ist.

Regel 12.2.3: Die Mikroglosse sollte so kurz wie möglich sein, während die Eindeutigkeit erhalten bleibt.

Regel 12.2.4: Keine zwei kanonischen IDs dürfen sich nur in der Groß-/Kleinschreibung unterscheiden.

12.3 Reservierte IDs

ID	Zweck
<code>sgf:xx.self.n.self</code> <code>_reference</code>	Wird in Synapsen verwendet, um sich auf den Knoten selbst zu beziehen

<code>sgf:xx.unk.n.unknow</code>	Wird verwendet, wenn ein Teilnehmer unbekannt ist
<code>sgf:xx.null.n.null_reference</code>	Wird verwendet, wenn ein Teilnehmer absichtlich abwesend ist
<code>sgf:xx.any.n.any_reference</code>	Wird in Abfragemustern als Wildcard verwendet

Abschnitt 13: Ereignisquellen und Autorität

13.1 Zweck

Ereignisse in der perdurantistischen Schicht sind keine Deklarationen metaphysischer Wahrheit. Sie sind **verankerte Ankerpunkte** – etablierte Konsensidentifikatoren, die aus autoritativen Referenzen stammen. Dieser Abschnitt definiert, wie Quellen aufgezeichnet werden, was als autoritativ gilt und wie Quellmetadaten strukturiert sind.

13.2 Autoritative Quellen

Die folgenden gelten als autoritative Quellen für Synapedia-Ereignisse:

- Wikipedia-Artikel und Infoboxen (für historische Persönlichkeiten, Artefakte, Ereignisse)
- Wikidata-Eigenschaften und -Aussagen (für strukturierte Daten)
- Akademische Taxonomien und Ontologien (für biologische Arten, chemische Verbindungen)
- Standardreferenzwerke (Wörterbücher, Enzyklopädien), wo sie Konsens darstellen

13.3 Struktur der Quellmetadaten

Jeder perdurantistische Eintrag KANN einen `sourcing` -Block enthalten:

```

"sourcing": {
  "events": [
    {
      "event": "sgf:en.transport.v.carry_cargo",
      "source": "https://en.wikipedia.org/wiki/Wagon",
      "accessed": "2026-06-28",
      "confidence": "high"
    }
  ],
  "global_note": "Alle Ereignisse stammen, sofern nicht anders angegeben"
}

```

Felder:

Feld	Typ	Erforderlich	Beschreibung
events	Objekt []	Ja	Ereignis-spezifische Quellaufzeichnungen
global_note	String	Nein	Hinweis, der für alle Ereignisse gilt

Jeder Ereignisdatensatz:

Feld	Typ	Erforderlich	Beschreibung
event	String	Ja	Kanonische ID des Ereignisverbs
source	String	Ja	URI oder Zitat der Quelle
accessed	Datum	Ja	Datum des Zugriffs auf die Quelle
confidence	Enum	Ja	high, medium oder low

13.4 Quellrichtlinien

- Für physische Objekte und Artefakte: Wikipedia-Artikel, Standardreferenz.

- Für historische Persönlichkeiten: Wikipedia-Infobox (Geburtsdatum, Sterbedatum, Beruf, bemerkenswerte Leistung).
- Für biologische Arten: etablierte taxonomische Datenbanken (z.B. GBIF, ITIS).
- Für abstrakte Konzepte: Konsensdefinitionen aus autoritativen Wörterbüchern oder Ontologien.
- Für fiktive Entitäten: das Ursprungsveröffentlichungsereignis (z.B. *A Study in Scarlet*, 1887, London).

13.5 Aktualisierung von Quellen

Wenn eine Quelle aktualisiert oder eine bessere Quelle verfügbar wird, wird der Synapedia-Eintrag über Deprecation und Ersetzung aktualisiert (Abschnitt 8). Die Quellmetadaten werden aktualisiert, um die neue Autorität widerzuspiegeln.

Abschnitt 14: Verankerung und Validierung

14.1 Verankerungsüberprüfungsalgorithmus

1. Wenn der Knoten im Prime Registry ist, gib GROUNDED zurück.
2. Wenn der Knoten eine angehängte Raumzeit-Koordinate hat, gib GROUNDED zurück.
3. Führe BFS entlang aller ausgehenden **IS-A** -Kanten durch.
4. Wenn irgendein erreichbarer Knoten im Prime Registry ist oder eine Raumzeit-Koordinate hat, gib GROUNDED zurück.
5. Andernfalls gib UNGROUNDED zurück.

14.2 Azyklizitätsüberprüfung

Führe eine topologische Sortierung des gesamten IS-A-Graphen durch. Wenn sie erfolgreich ist, bestehen. Wenn sie fehlschlägt (Zyklus erkannt), fehlschlagen und den Zyklus melden.

14.3 Mikroglossenhinlänglichkeitsüberprüfung

Gruppieren Sie alle Knoten nach (language_tag, lemma, pos). Überprüfen Sie für jede Gruppe, dass alle Mikroglossen eindeutig sind. Wenn ein Duplikat gefunden wird, fehlschlagen Sie mit dem Duplikatpaar.

14.4 Ereignisminimalitätsüberprüfung

Für jede Synapse in `essential_events` eines perdurantistischen Knotens:

1. Entfernen Sie die Synapse vorübergehend.
2. Vergleichen Sie die reduzierte Menge mit jedem Lemma-Geschwister.
3. Wenn die reduzierte Menge mit den wesentlichen Ereignissen eines Lemma-Geschwisters identisch ist, dann ist die entfernte Synapse notwendig – sie muss behalten werden.
4. Wenn die reduzierte Menge nicht mit den wesentlichen Ereignissen eines Lemma-Geschwisters identisch ist, dann ist die entfernte Synapse unnötig – fehlschlagen Sie.

14.5 Semantik der Verankerung – Entwurfshinweis

Verankerung ist eine **referentielle Garantie**, keine Wahrheitsgarantie. Ein verankerter Knoten hat eine definite Adresse im Graphen und einen Pfad zu Primitiven oder Raumzeit. Sie garantiert weder die reale Existenz (fiktionalen Entitäten werden über einen `fictional_entity`-Zweig verankert) noch, dass irgendeine Aussage, die das Konzept verwendet, wahr ist (Wahrheit gehört zur ABox). Verankerung garantiert nur, dass das Konzept nicht in Begriffen undefinierter Konzepte definiert ist – kein infinites Regress, keine schwebenden Definitionen.

Abschnitt 15: Wissenspakete

15.1 Struktur

Feld	Typ	Erforderlich	Beschreibung
<code>pack_id</code>	String	Erforderlich	Global eindeutiger Identifikator
<code>version</code>	String	Erforderlich	Semantische Version
<code>signature</code>	String	Erforderlich	Digitale Signatur
<code>authority</code>	String	Erforderlich	Signierende Entität
<code>references</code>	String[]	Erforderlich	Array kanonischer IDs
<code>content</code>	Objekt[]	Erforderlich	Array von SGF-Objekten

15.2 Beziehung zu Synapedia

Einschränkung 15.2.1: Ein Wissenspaket darf keine Knoten enthalten, die behaupten, in einer Synapedia-Schicht zu sein.

Einschränkung 15.2.2: Ein Wissenspaket darf keine `IS-A` -Kanten zum Synapedia-Graphen hinzufügen.

Einschränkung 15.2.3: Ein Wissenspaket darf zusätzliche Synapsen unter Verwendung derselben 15-Rollen-Grammatik definieren.

Einschränkung 15.2.4: Ein Wissenspaket darf jede gültige kanonische ID referenzieren, einschließlich veralteter IDs.

Abschnitt 16: Umfang und Grenzen

16.1 Was Synapedia bereitstellt

1. Maschinenadressierbare Konzeptdefinitionen mit eindeutigen, stabilen Identifikatoren.

2. Eine verankerte Typhierarchie (IS-A), die zu Primitiven oder Raumzeit zurückverfolgt.
3. Kompositionelle Struktur (Mereologie), wo tragend.
4. Minimale Ereignisdefinitionen, die Konzepte von Lemma-Geschwistern unterscheiden.
5. Eine geschlossene Grammatik von 15 semantischen Rollen für die Ereignisrepräsentation.
6. Ein Referenzsubstrat, auf das andere Schichten ohne Mehrdeutigkeit verweisen können.
7. Verankerte Ereignisanker, die Konsensdefinitionen repräsentieren, nicht ultimative Wahrheit.

16.2 Was Synapedia nicht bereitstellt

1. **Wahrheitswerte für Propositionen.** Wahrheit ist der Bereich der ABox.
2. **Temporales Schließen über Ereignisordnung hinaus.** Das ist der Bereich der Inferenzschicht.
3. **Probabilistisches oder unsicheres Wissen.** Synapedia-Einträge sind kategorial.
4. **Normatives oder deontisches Schließen.** Das ist der Bereich der Wissenspakete.
5. **Natürlichsprachliche Generierung oder Analyse.** Synapedia ist kein Sprachmodell.
6. **Enzyklopädisches Wissen.** Synapedia ist von Natur aus minimal.
7. **Inferenz oder Deduktion.** Das ist der Bereich der Reasoning-Schicht.

16.3 Grenze zur ABox

Die ABox verwendet kanonische IDs von Synapedia, um auf Konzepte zu verweisen. Die Beziehung ist einseitig: Die ABox referenziert Synapedia; Synapedia referenziert die ABox nicht. Behauptungen in der ABox können widersprüchlich, falsch oder bewusst täuschend sein. Synapedia löst dies nicht auf.

16.4 Grenze zu Wissenspaketen

Wissenspakete fügen Synapedia-Knoten zusätzliche Struktur hinzu. Sie sind signiert, versioniert und domänenspezifisch. Sie sind nicht Teil von Synapedia.

Abschnitt 17: Prüftiefe als politische Entscheidung

17.1 Das Problem

Synapedia definiert Konzepte mit mehreren Strukturschichten, aber wie tief ein nachgelagertes System zwei Einträge vergleicht, wird nicht vom Lexikon festgelegt. Es ist eine **politische Entscheidung**, die von den Folgen eines Fehlers bestimmt wird.

Jeder Vergleich beginnt mit derselben Grundlage: **Lemma + Wortart + Embedding**. Das Embedding bietet eine Position im mehrdimensionalen Vektorraum der Bedeutung. Es ist das universelle Adresssystem – es funktioniert sprachübergreifend, lexikonübergreifend und ontologieübergreifend, selbst wenn keine kanonischen IDs geteilt werden. Ohne das Embedding könnten zwei Systeme, die unterschiedliche Lexika verwenden, nicht einmal die Konzepte des jeweils anderen finden.

Die Frage ist: Sobald das Embedding uns in die richtige Nachbarschaft gebracht hat, wie viel strukturelle Überprüfung benötigen wir, bevor wir sicher genug sind zu handeln?

17.2 Die Rolle von Embeddings auf allen Tiefen

Auf jeder Tiefe beginnt der Vergleich mit denselben drei Signalen:

- **Lemma** – filtert auf die exakte Wortform (oder ihr sprachübergreifendes Äquivalent über TRANSLATION-OF -Kanten).
- **Wortart** – filtert auf die korrekte Verwendung (Nomen vs. Verb vs. Adjektiv).

- **Embedding** – liefert eine Position im Vektorraum, die Synonymie, Domäne und Verwendungsmuster erfasst.

Das Lemma ist beim Sprachwechsel nicht nutzbar (ein französisches Wort und ein englisches Wort haben unterschiedliche Lemmata), aber das Embedding funktioniert sprachübergreifend, da es auf mehrsprachigen Korpora trainiert ist. Für die meisten gewöhnlichen Suchen – Finden einer äquivalenten Wortbedeutung – sind Lemma + POS + Embedding ausreichend.

Die tieferen Schichten (Ontologisch, Mereologisch, Perdurantistisch) ersetzen das Embedding nicht. Sie **ergänzen** es. Sie liefern strukturelle Evidenz, die das Embedding allein nicht garantieren kann. Durch tieferes Eindringen in Synapedia – Untersuchen der Elternbegriffe, der Teile, der Ereignisse und der Eltern der Eltern – gewinnen wir viel größeres Vertrauen, dass zwei Konzepte wirklich äquivalent sind, nicht nur benachbart im Vektorraum.

17.3 Drei natürliche Tiefen

Tiefe 1 — Lemma + Embedding (Gewöhnliche Suche)

Vergleiche nur die lexikalische Schicht: Lemma, Wortart und Embedding-Vektor. Schnell, billig, geeignet für Erkundung und beiläufiges Stöbern. Das ist es, was die gewöhnliche menschliche Konversation meistens tut – wir hören ein Wort, unser Gehirn aktiviert das nächstgelegene Konzept über distributionelle Ähnlichkeit, und wir machen weiter. Es funktioniert, weil die Folge von Mehrdeutigkeit gering ist; wir können im nächsten Satz klären.

Beispiel: Ein Kunde, der in einem Hardware-Katalog stöbert, tippt „Schraubendreher“ ein. Das System gibt alle Schraubendreher-SKUs zurück. Wenn der Kunde „Kreuzschlitz“ meinte und das System „Schlitz“ zurückgibt, sind die Kosten eine kleine Korrektur.

Tiefe 2 — Struktureller Abgleich (Eine Ebene tief)

Vergleiche die unmittelbaren ontologischen, mereologischen und perdurantistischen Schichten Slot für Slot. Das System untersucht:

- IS-A-Eltern (was für eine Art Ding ist das?)
- HAS-COMPONENT-Teile (woraus besteht es?)
- Wesentliche Ereignisse (was tut es?)
- HAS-ATTRIBUTE-Werte (welche Eigenschaften hat es?)

Jeder besetzte Slot muss übereinstimmen. Leere Slots sind Wildcards. Diese Tiefe ist erforderlich für Kaufverpflichtungen, sicherheitskritische Teile und die meisten Beschaffungsvorgänge.

Beispiel — Der NASA-Schraubendreher: Ein Beschaffungsbeamter benötigt einen flugtauglichen Drehmomentschraubendreher – die Art, die auf der Mars-Mission verwendet wird. Ein Anbieter bietet einen Titan-Drehmomentschraubendreher mit rot eloxiertem Kragen an. Tiefe 1 (Lemma + Embedding) würde „Drehmomentschraubendreher“ mit „Drehmomentschraubendreher“ abgleichen und einen Kandidaten zurückgeben. Aber Tiefe 2 zeigt, dass die Anforderung `HAS_ATTRIBUTE: operating_environment = vacuum` und `HAS_REASON: use_case = aerospace_assembly` hat, während das angebotene Teil `HAS_ATTRIBUTE: operating_environment = atmospheric` hat. Der GapReport dokumentiert die Abweichung. Der Beamte rät nicht. Das System beweist den Unterschied.

Beispiel — Ausschreibung mit 200 Positionen: Eine Beschaffungsabteilung gibt eine Ausschreibung mit 200 Positionen heraus, die jeweils erforderliche Merkmale, Zertifizierungen und Leistungsschwellen spezifizieren. Der Tiefe-2-Abgleich läuft gleichzeitig für jede Position und erzeugt eine Compliance-Matrix. Positionen, die alle Slots bestehen, erhalten einen ProofTrace. Positionen, die durchfallen, erhalten einen GapReport, der genau zeigt, welcher Slot durchgefallen ist. Menschliche Überprüfung ist nur für die Ausnahmen erforderlich.

Tiefe 3 — Vollständige Hierarchiedurchquerung (Eltern der Eltern)

Durchquere zwei oder mehr Generationen von IS-A-Eltern. Vergleiche nicht nur die unmittelbaren Eltern, sondern auch die Eltern der Eltern. Dies erfasst Fälle, in denen der unmittelbare Elternteil nicht übereinstimmt, aber der Großelternteil schon.

Beispiel: „Schlaghosen“ stimmen möglicherweise nicht direkt mit „Hosen“ überein (unterschiedliche Mikroglosse), aber beide sind `IS-A` `unteres_Körperkleidungsstück`. Tiefe 3 findet die Verbindung, die Tiefe 2 übersehen würde.

Tiefe 3 wird selten für die Produktidentifikation benötigt – die definitorischen Ereignisse auf Eintragungsebene reichen normalerweise aus. Aber sie wird wichtig, wenn:

- Das Lexikon dünn besetzt ist (wenige Einträge existieren für die Domäne)
- Die Domäne tief verschachtelte Hierarchien hat (biologische Taxonomie, militärische Spezifikationen)
- Zwei Systeme unterschiedliche Granularität in ihren Ontologien verwenden (eines hat „Fahrzeug“ als Elternteil, das andere „Landfahrzeug“)

17.4 Politik, nicht Technologie

Wie tief sollten wir gehen? Es ist eine **politische Entscheidung**, keine technische Einheitslösung. Dasselbe System kann unterschiedliche Tiefen für unterschiedliche Kontexte durchsetzen:

- Ein Luxusuhrenhändler könnte für jeden Kauf Tiefe 2 festlegen, weil die Rückgabekosten hoch sind.
- Ein Lebensmittellieferdienst könnte für die meisten Artikel bei Tiefe 1 bleiben und Tiefe 2 nur für Artikel mit Allergen- oder Diätbeschränkungen reservieren.
- Ein Rüstungsunternehmen könnte Tiefe 3 für jedes Teil verlangen, das an Fluggeräte kommt.

Die Politik muss **transparent veröffentlicht** werden an alle Parteien in einem Verifikationsaustausch. Der Kunde (oder Beschaffungsbeamte) muss wissen, welche Tiefe verwendet wurde und was sie garantiert.

Einschränkung 17.4.1: Die Tiefenpolitik muss transparent an alle Parteien in einem Verifikationsaustausch veröffentlicht werden.

Einschränkung 17.4.2: Die Tiefenpolitik muss für jeden Vergleich im ProofTrace oder GapReport aufgezeichnet werden.

17.5 Zusammenfassungstabelle

Tiefe	Was wird verglichen	Geschwindigkeit	Vertrauen	Wann verwenden
1	Lemma + POS + Embedding	Schnell (~200ms)	Niedrig	Stöbern, Erkundung, risikoarme Abfragen
2	Unmittelbare ontologische , mereologische, perdurantistische Schichten	Mittel (~500ms)	Hoch	Kaufverpflichtungen, sicherheitskritische Teile, Ausschreibungsabgleich
3	Vollständige Hierarchiedurchquerung (Eltern der Eltern)	Langsamer (~1-2s)	Sehr Hoch	Dünn besetzte Lexika, tiefe Taxonomien, sprachübergreifender Abgleich

Abschnitt 18: Verankerung nicht-physischer Entitäten

18.1 Fiktive Entitäten

Fiktive Entitäten existieren nicht in der physischen Realität, benötigen aber dennoch verankerte Definitionen.

Verankerungsstrategie:

1. IS-A-Kette: `fictional_character.n.imaginary_person` IS-A
`fictional_entity.n.imaginary_thing` IS-A
`abstract_entity.n.non_physical_thing`
2. Ursprungs-Raumzeit-Koordinate: das Veröffentlichungsdatum und der Ort des Werkes, in dem die Figur erstmals erschien.

Beispiel — Sherlock Holmes:

- IS-A: `fictional_character.n.imaginary_person`
- Raumzeit-Koordinate: (1887, London) — Veröffentlichung von *A Study in Scarlet*
- Quellenangabe: Conan-Doyle-Kanon, autoritative Wissenschaft

Dies ist keine Behauptung, dass Holmes in London existierte. Es ist eine Behauptung, dass seine Definition dort ihren Ursprung hat.

18.2 Abstrakte Konzepte

Abstrakte Konzepte (Gerechtigkeit, Freiheit, Liebe, Schwerkraft) werden verankert über:

1. IS-A-Elternteil: `abstract_entity.n.concept`
2. Konsensdefinition: entnommen aus autoritativen Wörterbüchern, Enzyklopädien oder philosophischen Referenzwerken
3. Perdurantistische Ereignisse: nur bei Bedarf zur Disambiguierung

18.3 Mathematische Objekte

Mathematische Objekte (Zahlen, Mengen, Funktionen) werden verankert über:

1. IS-A-Elternteil: `abstract_entity.n.mathematical_object`
2. Formale Definition: entnommen aus standardmäßigen mathematischen Referenzen
3. Keine Raumzeit-Koordinate – mathematische Objekte haben keinen physischen Ursprungspunkt

Abschnitt 19: Vergleich mit bestehenden Systemen

System	Verankert	Geschlossene Grammatik	Minimalität	IS-A-Hierarchie	Ere
WordNet	Nein	Ja (eingeschränkt)	Nein	Ja (flach)	Nein
Wikidata	Teilweise	Nein	Nein	Ja (über P31/P279)	Nein
FrameNet	Nein	Nein	Nein	Nein	Teil
Cyc	Nein	Nein	Nein	Ja	Nein
DBPedia	Ja (URI)	Nein	Nein	Teilweise	Nein
Synpedia	Ja	Ja (15 Rollen)	Ja (Axiom V)	Ja (azyklisch)	Ja (Pe h)

Wesentliche Unterscheidungsmerkmale:

- Synapedia ist das einzige System, das Ereignisse als Teil der Definition verlangt.
- Synapedia ist das einzige System mit einer geschlossenen, begrenzten Prädikatenmenge.
- Synapedia erzwingt Minimalität algorithmisch – kein anderes System tut dies.
- Synapedia ist das einzige System, das Definition (TBox) explizit von Behauptung (ABox) trennt.

Prädikatenexplosion ist das systemische Versagen, das auftritt, wenn eine Ontologie eine unbegrenzte Menge von Beziehungstypen zulässt. In Systemen mit offenen Prädikatenräumen (Wikidata, RDF/OWL) kann jeder Entwickler neue Eigenschaften erfinden. Dies verursacht Schemafragmentierung, verlangsamt das Schließen und führt zu graphübergreifender Inkompatibilität. Synapedia verhindert Prädikatenexplosion, indem es relationale Kanten dauerhaft auf genau 15 invariante semantische Rollen beschränkt (Postulat III — Die Invariante Kantenbeschränkung).

Abschnitt 20: Beispiele

20.1 Wagen

Kanonische ID: `sgf:en.wagon.n.horse_drawn_cargo`

Vollständiger Eintrag, der alle vier Schichten durchspielt, enthält Mereologie, Polyhierarchie (Fahrzeug + Behälter) und mehrere Ereignisse. Demonstriert, dass Mereologie nur dann gehört, wenn Komposition tragend ist.

20.2 Beethoven

Kanonische ID: `sgf:en.beethoven.n.composer_1770`

Eigenname für ein einzigartiges Individuum. Minimale unterscheidende Ereignisse (genau drei):

1. Geburt: `HAS_TIME: 1770` , `HAS_LOCATION: Bonn`
2. Tod: `HAS_TIME: 1827` , `HAS_LOCATION: Wien`
3. Autor: `HAS_THEME: sgf:en.symphony_no_9.n.beethoven_op_125`

Diese drei Synapsen reichen aus, um diesen Beethoven von jeder anderen Entität namens „Beethoven“ im Lexikon zu unterscheiden. Keine Biographie ist nötig. Enzyklopädische Tiefe wird an Wissenspakete delegiert. Leere Mereologie ist gültig. Doppelt verankert (Prime Registry + Raumzeit).

20.3 Tomate

Kanonische ID: `sgf:en.tomato.n.edible_red_fruit`

Polyhierarchie aus unterschiedlichen Perspektiven (botanische Frucht + kulinarisches Gemüse). Leeres `essential_events` ist gültig, wenn die Mikroglosse ausreicht. Mereologie ist für organische Entitäten natürlich.

20.4 Bank (Finanzinstitut vs. Flussufer)

IDs: `sgf:en.bank.n.financial_institution` und `sgf:en.bank.n.river_edge`

Lemma-Geschwister-Disambiguierung über Mikroglosse + Ereignisse. Radikal unterschiedliche IS-A-Eltern. Der Säuretest für Homonymiebehandlung.

20.5 Amphibienfahrzeug

ID: `sgf:en.amphibious_vehicle.n.land_and_water`

Konfliktlösung für vererbte Ereignisse. Die meisten Polyhierarchien erzeugen keine echten Konflikte; beide Ereignisse werden ohne Probleme vererbt. Demonstriert explizite Überschreibung bei Bedarf.

20.6 Hypothek

ID: `sgf:en.mortgage.n.property_loan_agreement`

Abstraktes Konzept, vollständig durch Ereignisse definiert (erstellen, verpfänden, zurückzahlen, zwangsvollstrecken). Keine physische Verankerung; keine Mereologie. Demonstriert, dass abstrakte Konzepte vollständig durch ihren Ereignislebenszyklus definiert werden können.

20.7 Theodore Roosevelt (mehrere)

IDs: `sgf:en.theodore_roosevelt.n.president_1858` ,
`sgf:en.theodore_roosevelt.n.industrialist_1831` , etc.

Demonstriert die Disambiguierung von Eigennamen durch minimale verankerte Ereignisse (Geburt, Tod, Beruf, bemerkenswerte Leistung).
Quellenangabe aus Wikipedia-Infoboxen.

Abschnitt 21: Proof-of-Concept-Durchlauf

21.1 Proposition: „Beethoven komponierte die 9. Sinfonie im Jahr 1824.“

Schritt 1 — Nachschlagen: Die ABox enthält eine Synapse, die

`sgf:en.compose.v.create_music` referenziert mit `HAS_AGENT:`
`sgf:en.beethoven.n.composer_1770` , `HAS_THEME:`
`sgf:en.symphony_no_9.n.beethoven_op_125` , `HAS_TIME: "1824"` .

Schritt 2 — Synapedia-Verifikation: Die perdurantistische Schicht Beethovens enthält eine passende Synapse. Die perdurantistische Schicht der 9. Sinfonie enthält eine passende Synapse (mit vertauschten Rollen).

Schritt 3 — Validierung: Alle Axiome sind erfüllt:

- Axiom I: Keine Zyklen in IS-A-Pfaden.
- Axiom II: Beide Knoten haben Pfade zum Prime Registry. Beethoven hat zusätzlich Raumzeit-Koordinaten.
- Axiom IV: Mikroglossen sind eindeutig.

- Axiom V: Das Entfernen einer einzelnen Synapse von Beethoven würde die Disambiguierung nicht verlieren (kein anderes Lemma-Geschwister existiert), aber wenn es einen zweiten identischen Eintrag gäbe, würde die Minimalitätseinschränkung greifen.
- Axiom VI: Keine ID-Kollisionen.

Schritt 4 — Abfrage: „Wer komponierte die 9. Sinfonie?" → Rufe die perdurantistische Schicht der Sinfonie ab, finde die `compose` -Synapse, lese `HAS_AGENT` . → Gib „Beethoven" zurück.

Schritt 5 — Grenze: „Schrieb Beethoven andere Sinfonien?" → Kann nicht allein aus Synapedia beantwortet werden. Erfordert ein Wissenspaket mit vollständigem Katalog.

Abschnitt 22: Anhänge

Anhang A: Bootstrap-Plan

Synapedia kann nicht auf einmal aufgebaut werden. Der folgende phasenweise Plan definiert die minimal lebensfähige Menge von Einträgen.

Phase 0: Das Prime Registry (50 Einträge)

Kandidaten (angepasst von NSM-Primitiven und zusätzlichen Primitiven):

- Selbst/Andere: `i.n.self_reference` , `you.n.interlocutor` , `someone.n.person` , `something.n.entity`
- Handlungen: `do.v.perform_action` , `happen.v.occurevent` , `move.v.change_location` , `make.v.create_thing`
- Qualitäten: `good.adj.desirable` , `bad.adj.undesirable` , `big.adj.large_size` , `small.adj.little`
- Beziehungen: `before.preptime_sequence` , `after.preptime_sequence` , `because.conj.causal_link`

- Zeit/Raum: `time.n.dimension` , `place.n.location` ,
`now.n.present_moment` , `here.n.this_place`
- Logik: `not.conj.negation` , `maybe.conj.possibility` , `can.n.ability`

Phase 1: Hochrangige Kategorien (100 Einträge)

`person.n.human_individual` , `animal.n.living_creature` ,
`object.n.physical_thing` , `artifact.n.human_made` , `event.n.occurrence` ,
`action.n.deliberate_act` , `state.n.condition` , `place.n.location` ,
`time.n.interval` , `relation.n.connection` .

Phase 2: Kernverben und Ereignistypen (200 Einträge)

Bewegung, Schöpfung, Kommunikation, Besitz, Wahrnehmung, Kognition, Kernzustände. Jedes Verb definiert mit seinen Rolleneinschränkungen (`core_roles`, `permitted_roles`).

Phase 3: Konkrete Objekte (300 Einträge)

Körperteile, Werkzeuge, Fahrzeuge, Gebäude, Kleidung, Nahrung, Möbel. Jedes mit minimalen perdurantistischen Ereignissen und Mereologie, wo tragend.

Phase 4: Abstrakte Konzepte (200 Einträge)

Institutionen, Beziehungen, Vereinbarungen, Emotionen, Quantitäten, Eigenschaften.

Phase 5: Eigennamen (100 Einträge)

Bedeutende historische Persönlichkeiten, Orte, Kunstwerke.

Gesamtes Anfangsziel: ungefähr 1.000 Einträge.

Einschränkung A.1: Kein Eintrag in einer späteren Phase darf einen Eintrag aus einer späteren Phase referenzieren. Abhängigkeiten müssen die Phasenordnung respektieren.

Anhang B: Reservierte IDs

Wie in Abschnitt 12.3 spezifiziert.

Abschnitt 23: Referenzen

- Wierzbicka, A. (1996). *Semantics: Primes and Universals*. Oxford University Press.
- Goddard, C. (2018). *Ten Lectures on Natural Semantic Metalanguage*. Brill.
- Miller, G. A. (1995). "WordNet: A Lexical Database for English." *Communications of the ACM*, 38(11), 39-41.
- Baker, C. F., Fillmore, C. J., & Lowe, J. B. (1998). "The Berkeley FrameNet Project." *Proceedings of COLING-ACL*.
- Lenat, D. B. (1995). "Cyc: A Large-Scale Investment in Knowledge Infrastructure." *Communications of the ACM*, 38(11), 33-38.
- Vrandečić, D., & Krötzsch, M. (2014). "Wikidata: A Free Collaborative Knowledge Base." *Communications of the ACM*, 57(10), 78-85.
- Symbol Grounding Framework (2025-2026). *SGF Architecture Specification Series*. SGF-ARC.

End of Synapedia Architecture Specification v1.3